



重载

▼ 函数重载

函数重载是指在同一个作用域内定义多个具有相同名字、但参数列表不同的函数。C++ 通过函数签名（函数名 + 参数类型和数量）来区分这些函数，从而根据传入的参数类型和数量来调用正确的函数版本。

函数重载的规则：

- 函数名相同。
- 参数列表不同：参数的类型、数量或顺序不同。
- 参数相同但返回值类型不同的同名函数并不构成重载，因为编译器并不知道你想要的返回值类型，所以无法做出决策。这种情况推荐使用**模板**或**显示类型转换**。

▼ 构造函数重载

构造函数重载是函数重载的特殊形式，专门用于类的构造函数。通过重载构造函数，你可以为类定义多个构造函数，允许对象根据不同的初始化方式进行构造。

▼ 模板重载

详情见[模板](#)。

▼ 运算符重载 operator

运算符分为内置运算符与自定义运算符。

内置运算符用于内置类型（如 `int`，`float`，`double` 等）的运算。这些运算符的行为是固定的，由编译器内置定义。而为自定义的类或结构体重载的运算符即自定义运算符。

所以重载的运算符必须有自定义的类或结构体参与。

虽然重载运算符必须有自定义的类或结构体参与，但运算符重载函数并不一定要是其成员函数。

运算符重载函数也可以使用函数模板与函数重载。

`operator` 重载机制根据语义可以分为两种，一种是类似 `+` 只有一个符号，此类符号一般是二元运算符；另一种是类似 `()` 由左右两个符号组成的调用运算符，此类运算符的重载函数可以包含更多参数。两种实现的本质相同，但在使用起来有些许不同。

不过需要注意的是 `+` 这种二元运算符可以使用非成员函数形式，而类似 `()` 的调用运算符则一般不会使用非成员函数形式，这是因为此类符号需要与左操作数搭配使用，脱离左操作数使用时没有意义。

▼ 成员函数形式

运算符函数定义在类内时为成员函数，通过 `operator` 关键字将运算符转化为某种成员函数，操作符的左操作数隐式地成为调用运算符的对象。

```
▼ operator+ C++
class Vector {
public:
    int x, y;
    Vector(int x = 0, int y = 0) : x(x), y(y) {}
    // 重载+运算符
    Vector operator+(const Vector& v) {
        return Vector(x + v.x, y + v.y);
    }
};

int main() {
    Vector v1(1, 2), v2(3, 4);
    Vector v3 = v1 + v2;
    cout << "v3: (" << v3.x << ", " << v3.y << ")" << endl;
    return 0;
}
```

看似 `v1+v2` 有两个对象参与，但其实 `operator+` 只需要接受一个参数。

这是因为 `+` 只由左侧的 `v1` 触发。具体来说，`v1 + v2` 实际上是翻译成了 `v1.operator+(v2)`。

类似 `()` 由左右两个符号组成的符号，其使用方法与前者没有不同。`obj(args)` 实际上是 `obj.operator()(args)`。

```
▼ operator() C++
class Functor {
public:
    // 重载()运算符，接受两个整数参数并返回它们的和
    int operator()(int a, int b) {
        return a + b;
    }
};

int main() {
    Functor add; // 定义一个重载了()运算符的对象
    int result = add(5, 3); // 像调用函数一样使用该对象
    cout << "Result: " << result << endl; // 输出: 8
    return 0;
}
```

▼ 非成员函数形式

运算符函数定义在类外时为非成员函数，此时可以在类内声明为友元函数。

▼ 友元函数

运算符函数作为友元函数时，定义与成员函数略有不同，但使用起来基本相同：

```
C++

class MyClass {
public:
    int value;
    MyClass(int v) : value(v) {}
    // 友元函数重载 operator+
    friend MyClass operator+(const MyClass& lhs, const MyClass& rhs);
};

MyClass operator+(const MyClass& lhs, const MyClass& rhs) {
    return MyClass(lhs.value + rhs.value);
}

int main() {
    MyClass obj1(10);
    MyClass obj2(20);
    MyClass result = obj1 + obj2; // 使用友元函数重载的 operator+
    std::cout << "Result: " << result.value << std::endl; // 输出: Result: 3
    return 0;
}
```

```
C++

class MyClass {
public:
    friend void operator()(MyClass& obj, int x); // 友元函数声明
};

void operator()(MyClass& obj, int x) { // 友元函数定义
    std::cout << "Called with: " << x << std::endl;
}

int main() {
    MyClass obj;
    obj(10); // 通过对象直接调用友元 operator()
    return 0;
}
```

友元函数不同于成员函数，C++ 编译器不再自动将调用该函数的对象作为**隐式参数**传递给函数，所以在定义时需要声明一个**显式参数**来接受左操作数。

`obj1 + obj2` 会被解析为 `operator+(obj1, obj2)`，`obj(10)` 会被解析为 `operator()(obj, 10)`，左操作数会作为第一个**显式参数**。

▼ 非友元函数

如果运算符函数为非成员函数，并且为非友元函数，那么二元运算符与调用运算符，虽然均不能访问保护成员和隐私成员，但在能否隐式地调用运算符函数上出现了不同之处。

因为 `+` 是标准的运算符之一，编译器知道该如何寻找并解析它的重载，所以即使 `operator+` 依然可以隐式地调用，即 `obj1 + obj2`。

```
C++

class MyClass {
public:
    int value;
    MyClass(int v) : value(v) {}
};

// 非成员函数重载 operator+
MyClass operator+(const MyClass& lhs, const MyClass& rhs) {
    return MyClass(lhs.value + rhs.value);
}

int main() {
    MyClass obj1(10);
    MyClass obj2(20);
    // 使用非成员函数重载的 operator+ 进行隐式调用
    MyClass result = obj1 + obj2; // 编译器隐式调用 operator+(obj1, obj2)
    std::cout << "Result: " << result.value << std::endl;
    return 0;
}
```

但作为调用运算符的 `()`，它的作用是让对象可以像函数一样被调用，当它不与某个类相关联时，编译器不会自动将对象的调用解析为非成员 `operator()` 函数。

所以此时 `operator()` 函数不会被隐式调用，`obj(10)` 不会被解析为 `operator()(obj,10)`。若依然要使用该运算符函数，应该直接显式调用 `operator()(obj,10)`。

```
C++

class MyClass {
public:
    int value;
    MyClass(int v) : value(v) {}
};

// 非成员函数重载 operator()
void operator()(const MyClass& obj, int x) {
    std::cout << "Value: " << obj.value << ", x: " << x << std::endl;
}

int main() {
    MyClass obj(10);
    // 不能隐式调用非成员 operator()
    // obj(5); // 错误！非成员函数不能隐式调用
    // 只能显式调用
    operator()(obj, 5); // 显式调用非成员 operator()
}
```

▼ 类型转换函数

见 [【类型转换→隐式类型转换】](#)